Actividad de Laboratorio Semana 07

EJERCICIO 01: Adapter y Facade

ViajaFácil es una agencia de viajes. Se le solicita desarrollar un programa en Java que **registre reservas de paquetes turísticos, calcule el total a pagar de cada una** y consulte los cupos disponibles de cada destino en un sistema externo. Para ello, aplique el patrón **Adapter** (para comunicarse con el sistema externo) y el patrón **Facade** (para simplificar el uso del sistema).

Datos de la reserva

Campo	Tipo	Obligatorio
Código de reserva (ej. “RES-001”)	String	Sí
Destino (ej. “CUSCO”)	String	Sí
Cliente	String	Sí
Número de personas	int	Sí
Precio por persona en soles	double	No

Reglas

- Una reserva es válida si el código, el destino y el cliente no están vacíos y el número de personas es mayor que 0. Si no es válida, se rechaza y no se registra.
- Total de la reserva = número de personas × precio por persona.
- Si la reserva es de **4 personas o más** (reserva de grupo), se aplica un descuento del 10 % al total.

Sistema externo de cupos. Copie esta clase en su proyecto y **no la modifique**. Devuelve la información de un destino como un solo texto con el formato destino:cupos:temporada.

```
public class SistemaCuposExterno {
    private static final Map<String, String> CUPOS = Map.of(
        "CUSCO", "CUSCO:12:ALTA",
        "AREQUIPA", "AREQUIPA:8:MEDIA",
        "PARACAS", "PARACAS:20:BAJA");

    public String consultar(String destino) {
        return CUPOS.get(destino);
    }
}
```

Funcionamiento del programa (salida en consola)

1. Registrar las 5 reservas de prueba desde el Main. Cada reserva se valida y, si no es válida, se rechaza y se muestra un mensaje en la consola.
2. Calcular el total de cada reserva registrada (con el descuento de grupo cuando corresponda).
3. Consultar en el sistema externo los cupos disponibles del destino de cada reserva registrada.
4. Mostrar en la consola la lista de reservas registradas, ordenada por código, con su cliente, destino, total y cupos disponibles.
5. Mostrar la cantidad de reservas registradas, la suma de sus totales y la lista de las reservas de grupo (4 personas o más).

Datos de prueba

Registre exactamente estas reservas en su Main, en el orden indicado:

Código	Destino	Cliente	Personas	Precio por persona (S/)
RES-004	CUSCO	Ana Soto	2	450.00
RES-001	AREQUIPA	Luis Vega	5	300.00
RES-003	PARACAS	Rosa León	0	200.00
RES-002	CUSCO	Marta Gil	4	450.00
RES-005	PARACAS	Juan Pérez	3	180.00

Resultado esperado

Orden	Código	Cliente	Destino	Total (S/)	Cupos disponibles
1	RES-001	Luis Vega	AREQUIPA	1350.00	8
2	RES-002	Marta Gil	CUSCO	1620.00	12
3	RES-004	Ana Soto	CUSCO	900.00	12
4	RES-005	Juan Pérez	PARACAS	540.00	20

- RES-003 debe ser **rechazada** (0 personas) y debe verse el mensaje de rechazo en la consola.
- Cantidad de reservas registradas: **4**. Suma de totales: **S/ 4410.00**.
- Reservas de grupo (4 personas o más): **RES-001** y **RES-002**.

Qué debe hacer

1. Cree **una sola clase de modelo: Reserva**, con atributos privados y sus getters.
2. **Adapter:** cree la interfaz ProveedorCupos, con el método consultarCupos(destino), y la clase AdaptadorCupos. Esta clase implementa la interfaz, utiliza SistemaCuposExterno, separa el texto con split(":") y devuelve los cupos disponibles como número entero.
3. **Subsistemas:** cree ValidadorReserva (valida la reserva), CalculadoraPrecio (calcula el total con el descuento de grupo) y RegistroReservas (guarda las reservas).
4. **Facade:** cree SistemaReservas con las operaciones registrar reserva, validar reserva, calcular total y consultar cupos, delegando el trabajo en los subsistemas y en el adaptador.
5. **Main:** utilizando únicamente SistemaReservas, registre las 5 reservas de prueba. Luego, con **lambdas y streams (sin for/while)**, muestre la lista ordenada por código (con cliente, destino, total y cupos), la cantidad de reservas, la suma de sus totales y las reservas de grupo.

EJERCICIO 02: patrón MVC con Service

Enunciado

La academia Saber registra las notas de sus estudiantes. Se le solicita desarrollar una **aplicación en Java que permita registrar estudiantes y calcular su promedio y su condición (aprobado o desaprobado)**, con interfaz gráfica (Swing, con diseño libre). Para ello, aplique el patrón **MVC** y una capa **Service** que contenga las reglas de negocio.

Datos del estudiante

Campo	Tipo	Obligatorio
Código	String	Sí
Nombres	String	Sí
Nota 1	double	Sí
Nota 2	double	Sí
Nota 3	double	Sí

Reglas

- El código debe tener 6 dígitos y no puede repetirse.
- Los campos obligatorios no pueden estar vacíos.
- Cada nota debe estar entre 0 y 20.
- Promedio = (nota 1 + nota 2 + nota 3) / 3.
- Condición: **aprobado** si el promedio es 11 o más; **desaprobado** en caso contrario.
- Los errores se muestran en la ventana (por ejemplo, con un cuadro de diálogo), no por consola.

Funcionamiento de la aplicación (vista en Swing)

1. **Registrar un estudiante:** ingresar código, nombres y las tres notas. Si falta un dato, el código no tiene 6 dígitos o está repetido, o una nota está fuera de rango, la ventana muestra un mensaje de error.
2. **Ver los estudiantes registrados:** la ventana muestra la lista con código, nombres, promedio y condición de cada estudiante.
3. **Buscar un estudiante por código:** la ventana muestra sus datos, su promedio y su condición (o un mensaje si no existe).
4. **Ver el resumen del salón:** la ventana muestra el promedio general (promedio de los promedios), la cantidad de aprobados y la cantidad de desaprobados.

Datos de prueba

Registre estos estudiantes desde la interfaz, en el orden indicado:

Código	Nombres	Nota 1	Nota 2	Nota 3
202301	Ana Torres	14	16	15
202302	Luis Paredes	8	10	12
202301	Pedro Ruiz	12	12	12
202305	Rosa León	22	10	10
202303	Marta Gil	18	17	19
202304	Juan Pérez	10	11	12

Resultado esperado

Código	Nombres	Promedio	Condición
202301	Ana Torres	15.00	Aprobado
202302	Luis Paredes	10.00	Desaprobado
202303	Marta Gil	18.00	Aprobado
202304	Juan Pérez	11.00	Aprobado

- Pedro Ruiz debe ser **rechazado** (código repetido) y Rosa León debe ser **rechazada** (nota fuera de rango); ambos errores deben verse en mensajes de la ventana.
- Estudiantes registrados: **4**. Promedio general: **13.50**. Aprobados: **3**. Desaprobados: **1**.

Qué debe hacer

Organice el código en paquetes. El flujo es: **Vista → Controlador → Service → Modelo**.

Paquete	Clase	Qué debe contener
modelo	Estudiante	**Única clase de modelo.** Solo datos: atributos privados, constructor y getters.
service	EstudianteService	Guarda la lista de estudiantes y aplica las reglas (obligatorios, código de 6 dígitos y no repetido, notas entre 0 y 20). Calcula el promedio y la condición, busca por código y calcula el resumen del salón.
vista	VentanaEstudiantes	Solo componentes gráficos (campos, botones y tabla o área de texto). **Asigna los eventos de los botones con lambdas** y llama al controlador. Sin reglas de negocio.
controlador	EstudianteController	Recibe los datos de la vista, llama al service y devuelve los resultados ya preparados para mostrarse. **No conoce la vista.**
(raíz)	Main	Crea la vista, el service y el controlador, y abre la ventana.

1. Cree las clases indicadas en la tabla anterior.
2. En EstudianteService utilice **streams** para verificar si un código ya existe, buscar un estudiante por código, calcular el promedio general y contar los aprobados.

EVIDENCIAS

Pegue sus evidencias en los recuadros de cada ejercicio. En el **Ejercicio 01** la evidencia de ejecución es la **salida en consola** de NetBeans; en el **Ejercicio 02** es la **vista (ventana Swing)**. Recuerde que **cada captura debe mostrar el entorno completo de NetBeans, con la fecha y la hora visibles**.

Ejercicio 01 — ViajaFácil (Adapter y Facade): salida en consola

Evidencia 1.1 — Clase Reserva (modelo)

Pegue aquí la captura del código de la clase Reserva.

Evidencia 1.2 — Clase ProveedorCupos (interfaz del Adapter)

Pegue aquí la captura del código de la clase ProveedorCupos.

Evidencia 1.3 — Clase AdaptadorCupos (Adapter)

Pegue aquí la captura del código de la clase AdaptadorCupos.

Evidencia 1.4 — Clase ValidadorReserva (subsistema)

Pegue aquí la captura del código de la clase ValidadorReserva.

Evidencia 1.5 — Clase CalculadoraPrecio (subsistema)

Pegue aquí la captura del código de la clase CalculadoraPrecio.

Evidencia 1.6 — Clase RegistroReservas (subsistema)

Pegue aquí la captura del código de la clase RegistroReservas.

Evidencia 1.7 — Clase SistemaReservas (Facade)

Pegue aquí la captura del código de la clase SistemaReservas.

Evidencia 1.8 — Clase Main

Pegue aquí la captura del código de la clase Main.

Evidencia 1.9 — Ejecución (consola)

Pegue aquí la captura de la consola de NetBeans con la salida: mensaje de rechazo de RES-003, lista ordenada, cantidad, suma de totales y reservas de grupo.

Ejercicio 02 — Academia Saber (MVC con Service): vista en Swing

Evidencia 2.1 — Clase Estudiante (modelo)

Pegue aquí la captura del código de la clase Estudiante.

Evidencia 2.2 — Clase EstudianteService (service)

Pegue aquí la captura del código de la clase EstudianteService.

Evidencia 2.3 — Clase VentanaEstudiantes (vista)

Pegue aquí la captura del código de la clase VentanaEstudiantes.

Evidencia 2.4 — Clase EstudianteController (controlador)

Pegue aquí la captura del código de la clase EstudianteController.

Evidencia 2.5 — Clase Main

Pegue aquí la captura del código de la clase Main.

Evidencia 2.6 — Vista Swing: lista y resumen del salón

Pegue aquí la captura de la ventana Swing con los 4 estudiantes registrados, sus promedios y condiciones, y el resumen del salón.

Evidencia 2.7 — Vista Swing: errores de validación

Pegue aquí la captura de la ventana Swing mostrando el mensaje de error de Pedro Ruiz (código repetido) o de Rosa León (nota fuera de rango).